DriveSure

Vehicle Insurance Management System

Software Requirements Specification (SRS)

Version 1.0

March 2026

Project	DriveSure
Domain	Fintech / InsurTech
Document Type	SRS
Date	March 2026

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for DriveSure, a comprehensive Vehicle Insurance Management System. It is intended for developers, system architects, testers, and stakeholders to ensure a shared and precise understanding of the system to be built.

1.2 Scope

DriveSure automates the complete vehicle insurance lifecycle, covering:

- Policy browsing, application, and underwriting review
- Premium calculation using dynamic risk scoring
- Simulated payment processing
- Insurance claim filing and settlement workflow
- Role-based dashboards for Admin, Customer, Underwriter, and Claim Officer
- Real-time notification management
- Financial tracking, revenue reporting, and commission management

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token – used for stateless authentication
API	Application Programming Interface
REST	Representational State Transfer – architectural style for APIs
JPA	Java Persistence API – ORM specification
RxJS	Reactive Extensions for JavaScript – async event handling
RBAC	Role-Based Access Control
UI	User Interface
H2	In-memory relational database (for development/testing)
DXA	Document eXtended Attribute – unit for Word document measurements

1.4 References

- DriveSure Project Analysis Report (internal document)
- Spring Boot 4.x Documentation – https://spring.io/projects/spring-boot
- Angular 21 Documentation – https://angular.io
- JSON Web Token Standard – RFC 7519

- IRDAI (Insurance Regulatory and Development Authority of India) guidelines

1.5 Overview of Document

This SRS is organized as follows: Section 2 provides an overall system description; Section 3 enumerates functional requirements; Section 4 addresses non-functional requirements; Section 5 covers data modeling; Section 6 details system constraints; Section 7 presents interface requirements; and Section 8 outlines business rules.

2. Overall System Description

2.1 Product Perspective

DriveSure is a standalone web-based application that digitizes and automates vehicle insurance operations. It serves as a unified platform for all parties involved in the insurance lifecycle — customers purchasing policies, underwriters assessing risk, claim officers settling disputes, and admins overseeing the entire system. The system follows a client-server architecture with an Angular frontend consuming a Spring Boot REST API backed by a relational database.

2.2 Product Functions – High Level Summary

#	Function	Description
F1	User Registration & Authentication	Secure sign-up, login with JWT-based stateless sessions and RBAC
F2	Policy Management	CRUD operations for insurance policies with vehicle-type classification
F3	Quote & Premium Calculation	Risk-score-based premium computation before policy application
F4	Policy Application Lifecycle	Customer submission → Underwriter review → Approval/Rejection → Payment
F5	Claim Filing & Settlement	Customers file claims; Claim Officers review and approve/reject payouts
F6	Notification System	Real-time role-specific alerts for all system events
F7	Financial Dashboard	Admin revenue tracking, premium aggregation, and commission reporting

2.3 User Classes and Characteristics

2.3.1 ADMIN

System administrators with full platform access. They manage policies, monitor platform-wide revenue, view all notifications, and oversee commission payouts to staff. Assumed to have strong technical and domain knowledge.

2.3.2 CUSTOMER

End-users who register to purchase vehicle insurance. They browse policies, calculate premium quotes, submit applications, make payments, and file claims. Assumed to have basic digital literacy.

2.3.3 UNDERWRITER

Insurance risk professionals who review and approve or reject policy applications submitted by customers. They verify vehicle details, review risk scores, and are compensated via a 5% commission on approved premiums.

2.3.4 CLAIM_OFFICER

Claims settlement specialists who review claim requests, assess damages or injuries, validate supporting documents and garage estimates, and approve or reject payouts. They receive a 2% commission on associated premiums.

2.4 Operating Environment

Layer	Technology	Details
Frontend	Angular 21 (Standalone)	Reactive Forms, Tailwind CSS, RxJS, Route Guards
Backend	Spring Boot 4.x	Spring Security, JPA, Hibernate, REST Controllers
Database	H2 / MySQL	H2 for dev/test; MySQL for production
Authentication	JWT (io.jsonwebtoken)	Stateless, Bearer token-based session management
Communication	REST API	JSON over HTTPS between frontend and backend
Build Tools	Maven / Node.js + npm	Backend: Maven; Frontend: Angular CLI

2.5 Design and Implementation Constraints

- The system must use JWT for all authenticated endpoints.
- Backend must enforce role-based access at the API level, not just the UI level.
- Payment processing is simulated (no real payment gateway integration required in v1.0).
- Claims can only be filed against policies with status APPROVED and payment status PAID.
- The notification polling interval must not cause visible loading spinners in the UI.

2.6 Assumptions and Dependencies

- A valid MySQL instance is available for production deployment.
- All users access the application via modern web browsers (Chrome, Firefox, Edge).
- Vehicle age is derived from the Vehicle Year field provided at application time.
- The risk score formula (1.0–5.0 scale) is fixed and does not require external actuarial input in v1.0.

3. Functional Requirements

3.1 User Authentication & Security Module

3.1.1 User Registration

1. The system shall allow new users to register with a username, password, email, and phone number.
2. The default role assigned at registration shall be CUSTOMER.
3. Admin accounts shall be provisioned manually or via a seeded script.
4. Passwords shall be stored in a hashed format (BCrypt).

3.1.2 User Login

5. The system shall authenticate users using a username and password combination.
6. On successful login, the system shall return a signed JWT Bearer token.
7. The frontend shall store the JWT in localStorage and attach it as an Authorization header to all subsequent API calls.
8. Expired or invalid tokens shall result in a 401 Unauthorized response.

3.1.3 Route Guards & Authorization

9. Angular route guards shall prevent unauthorized access to role-specific dashboards.
10. The backend shall validate the JWT and enforce RBAC on all protected REST endpoints.
11. Users attempting to access resources beyond their role shall receive a 403 Forbidden response.

3.2 Policy Management Module

3.2.1 Policy Definition

12. Admin shall be able to create, update, and deactivate insurance policies.
13. Each policy shall have: Policy Name, Policy Type (Third-Party | Comprehensive), Vehicle Type (Car | Bike | Truck), Base Premium, and Coverage Amount.
14. Customers shall be able to browse all active policies filtered by vehicle type.

3.2.2 Risk Assessment & Quote Calculation

15. The system shall calculate a Risk Score in the range [1.0, 5.0] based on: vehicle age, vehicle type, and coverage amount.
16. Vehicles older than 15 years shall incur a risk score penalty of +0.8.
17. Total Premium = Base Premium * Risk Score. The calculated quote shall be displayed to the customer before application submission.

3.2.3 Policy Application Workflow

Step	Actor	Action	Resulting Status
1	Customer	Selects a policy and submits application	PENDING
2	Underwriter	Reviews vehicle details and risk score	—

Step	Actor	Action	Resulting Status
3	Underwriter	Approves the application	APPROVED
3b	Underwriter	Rejects the application	REJECTED
4	Customer	Completes payment (simulated gateway)	PAID

18. An application shall only transition to APPROVED status after Underwriter action.
19. Payment shall only be triggered after the application status is APPROVED.
20. A customer cannot resubmit a REJECTED policy application; a new application must be created.

3.3 Claim Settlement Module

3.3.1 Claim Filing

21. A customer shall only be able to file a claim against a policy subscription with status APPROVED and paymentStatus PAID.
22. The customer shall select a Claim Type: SELF (own vehicle/personal damage) or THIRD_PARTY (liability to a third party).
23. The customer shall specify an Injury Type: Minor Injury, Serious Injury, Death, or Vehicle Damage.
24. If Injury Type is Vehicle Damage, a Garage Estimate field shall be mandatory.
25. If Injury Type is a personal injury (Minor, Serious, Death), the Garage Estimate field shall be disabled.
26. The customer shall be able to attach supporting documents with the claim submission.

3.3.2 Claim Review

27. Claim Officers shall view a queue of all PENDING claims.
28. The Claim Officer shall review: claim reason, injury type, risk score, claim amount, and uploaded documents.
29. The Claim Officer shall approve or reject the claim, updating its status accordingly.
30. Third-party payout caps shall be enforced: Minor Injury = INR 50,000; Serious Injury = INR 2,00,000; Death = INR 5,00,000.

3.4 Notification System

31. A notification bell icon shall be permanently visible in the navigation bar for all logged-in users.
32. Unread notification count shall be displayed as a badge on the bell icon.
33. Notifications shall be fetched via background polling using RxJS interval, with no loading spinner shown during polling.
34. Admin shall receive all system-level notifications.
35. Customers shall receive notifications specific to their own policy applications and claims.
36. Underwriters shall receive alerts when new policy applications are submitted (PENDING).

37. Claim Officers shall receive alerts when new claims are filed (PENDING).
38. Marking a notification as read shall update its isRead flag via a PATCH API call.

3.5 Financial & Commission Module

39. Admin shall have access to a financial dashboard showing total premiums collected across all PAID subscriptions.
40. Upon policy approval and payment, the system shall automatically calculate and record:
    - Underwriter Commission = 5% of Total Premium
    - Claim Officer Commission = 2% of Total Premium (allocated upon claim assignment)
41. Admin shall be able to view a breakdown of commissions payable to individual Underwriters and Claim Officers.

4. Non-Functional Requirements

4.1 Performance

- API response time for standard CRUD operations shall be under 500ms on the production server.
- The notification polling interval shall be set to avoid UI flickering while ensuring near-real-time updates (recommended: 15–30 second interval).
- The system shall support at least 200 concurrent users without degradation in v1.0.

4.2 Security

- All API endpoints (except /register and /login) shall require a valid JWT.
- Passwords shall be stored using BCrypt hashing with a minimum cost factor of 10.
- The JWT secret key shall be stored in application configuration and not hardcoded in source code.
- HTTPS shall be enforced in production to prevent token interception.
- Role-based authorization shall be enforced at the Spring Security layer, independent of frontend guards.

4.3 Usability

- The application shall be fully functional on modern desktop web browsers (Chrome 100+, Firefox 100+, Edge 100+).
- Error messages shall be descriptive and user-friendly (e.g., 'Policy not found' rather than generic 500 errors).
- Form validations shall provide inline feedback to users before submission.

4.4 Reliability

- The system shall gracefully handle database connection failures with informative error responses.
- Notification polling failures shall not crash or freeze the UI.

4.5 Maintainability

- Backend code shall follow a layered architecture: Controller → Service → Repository.
- Frontend components shall use Angular standalone architecture for modularity.
- All business logic (risk calculation, commission computation) shall reside in the service layer, not controllers.

4.6 Scalability

- The database schema shall support migration from H2 to MySQL with no code changes to JPA entities.
- The system architecture shall permit horizontal scaling of the Spring Boot backend behind a load balancer.

5. Data Models

5.1 Entity: MyUser

Field	Type	Constraints	Description
id	Long	PK, Auto-generated	Unique user identifier
username	String	Unique, Not Null	Login username
password	String	Not Null	BCrypt-hashed password
roles	Set<Role>	Not Null	Assigned roles (CUSTOMER, ADMIN, etc.)
email	String	Unique, Not Null	User email address
phoneNumber	String	Optional	Contact number

5.2 Entity: Policy

Field	Type	Constraints	Description
policyId	Long	PK, Auto-generated	Unique policy identifier
name	String	Not Null	Policy display name
policyType	Enum	Not Null	THIRD_PARTY | COMPREHENSIVE
vehicleType	Enum	Not Null	CAR | BIKE | TRUCK
basePremium	Double	Not Null, > 0	Base annual premium amount (INR)
coverageAmount	Double	Not Null, > 0	Maximum coverage limit (INR)

5.3 Entity: PolicySubscription

Field	Type	Constraints	Description
id	Long	PK, Auto-generated	Unique subscription ID
policy	Policy	FK, Not Null	Reference to the Policy entity
user	MyUser	FK, Not Null	Reference to the Customer
status	Enum	Not Null	PENDING | APPROVED | REJECTED
paymentStatus	Enum	Not Null	PAID | PENDING
totalPremium	Double	Calculated	Base Premium * Risk Score
riskScore	Double	1.0–5.0	Calculated risk multiplier
commissions	Map/JSON	Optional	Commission records for UW and CO

5.4 Entity: Claim

Field	Type	Constraints	Description
id	Long	PK, Auto-generated	Unique claim ID
subscription	PolicySubscription	FK, Not Null	Associated policy subscription
claimAmount	Double	Not Null, > 0	Requested payout amount (INR)
claimType	Enum	Not Null	SELF | THIRD_PARTY
injuryType	Enum	Not Null	MINOR_INJURY | SERIOUS_INJURY | DEATH | VEHICLE_DAMAGE
garageEstimate	Double	Conditional	Mandatory if injuryType = VEHICLE_DAMAGE
status	Enum	Not Null	PENDING | APPROVED | REJECTED
documents	String/URL	Optional	Attached evidence / documents

5.5 Entity: Notification

Field	Type	Constraints	Description
id	Long	PK, Auto-generated	Unique notification ID
message	String	Not Null	Notification text content
role	String	Optional	Target role (for broadcast notifications)
userId	Long	Optional	Target user ID (for user-specific alerts)
isRead	Boolean	Default: false	Whether the notification has been read
createdAt	LocalDateTime	Auto-set	Timestamp when notification was created

6. System Constraints & Business Rules

6.1 Claim Constraints

- A claim can ONLY be filed if the associated PolicySubscription has status = APPROVED AND paymentStatus = PAID.
- Attempting to file a claim against a PENDING or REJECTED subscription shall return a 400 Bad Request error.
- The garage estimate field is required (not null) if and only if injuryType = VEHICLE_DAMAGE.

6.2 Risk Score Business Rules

Factor	Condition	Impact on Risk Score
Vehicle Age	Age > 15 years	+0.8 added to base risk score
Vehicle Type	Truck > Car > Bike	Trucks carry the highest base risk factor
Coverage Amount	Higher coverage	Marginally increases risk score
Risk Score Range	Always clamped	Minimum 1.0, Maximum 5.0

6.3 Third-Party Claim Payout Caps

Injury Type	Maximum Payout (INR)
Minor Injury	50,000
Serious Injury	2,00,000
Death	5,00,000
Vehicle Damage	As per garage estimate (no fixed cap)

6.4 Commission Rules

- Commissions are calculated and recorded at the time of policy payment confirmation.
- Underwriter Commission = 5% of the Total Premium of the approved policy.
- Claim Officer Commission = 2% of the Total Premium associated with the policy under which the claim is filed.
- Commissions are informational in v1.0; no automated payout mechanism is required.

6.5 UI Behavior Constraints

- The notification polling background job shall suppress the global loading spinner to avoid screen flickering.
- Route guards shall redirect unauthenticated users to the /login page.
- After logout, the JWT shall be cleared from localStorage immediately.

7. Interface Requirements

7.1 User Interface Requirements

- The application shall implement a responsive layout using Tailwind CSS utility classes.
- Each role shall have a dedicated dashboard accessible after login.
- A persistent navigation bar shall include: brand logo, navigation links relevant to the user's role, notification bell with unread badge, and a logout button.
- All forms shall use Angular Reactive Forms with synchronous and asynchronous validators.
- Loading states shall be indicated using spinners, excluding background notification polling.

7.2 API Interface Requirements

Module	Method	Endpoint Pattern	Description
Auth	POST	/api/auth/register	User registration
Auth	POST	/api/auth/login	User login, returns JWT
Policy	GET	/api/policies	List all active policies
Policy	POST	/api/policies	Admin: create new policy
Subscription	POST	/api/subscriptions	Customer: apply for a policy
Subscription	PATCH	/api/subscriptions/{id}/approve	Underwriter: approve application
Subscription	PATCH	/api/subscriptions/{id}/reject	Underwriter: reject application
Subscription	PATCH	/api/subscriptions/{id}/pay	Customer: confirm payment
Claim	POST	/api/claims	Customer: file a new claim
Claim	PATCH	/api/claims/{id}/approve	Claim Officer: approve claim
Claim	PATCH	/api/claims/{id}/reject	Claim Officer: reject claim
Notification	GET	/api/notifications	Fetch notifications for current user
Notification	PATCH	/api/notifications/{id}/read	Mark notification as read
Admin	GET	/api/admin/revenue	Admin: view total revenue & commissions

7.3 Database Interface

- Spring Data JPA repositories shall serve as the abstraction layer between the service layer and the relational database.
- Database schema shall be auto-generated by Hibernate (spring.jpa.hibernate.ddl-auto=update for dev; validate for prod).
- The application shall support H2 in-memory database for development and MySQL 8.x for production via Spring profiles.

8. Appendix

8.1 Use Case Summary

Use Case ID	Use Case Name	Primary Actor
UC-01	Register Account	Customer
UC-02	Login to System	All Roles
UC-03	Browse Insurance Policies	Customer
UC-04	Calculate Premium Quote	Customer
UC-05	Apply for Policy	Customer
UC-06	Review Policy Application	Underwriter
UC-07	Approve / Reject Policy	Underwriter
UC-08	Make Premium Payment	Customer
UC-09	File an Insurance Claim	Customer
UC-10	Review & Settle Claim	Claim Officer
UC-11	Manage Policies (CRUD)	Admin
UC-12	View Revenue Dashboard	Admin
UC-13	View Notifications	All Roles
UC-14	Mark Notification as Read	All Roles

8.2 Revision History

Version	Date	Author	Changes
1.0	March 2026	DriveSure Team	Initial SRS document created

— End of Document —